

Contents

1	Optimization Plan: ADER dynamic-rupture hot path (TPV31 spatial, $p \geq 2$ / ADER-O3)	1
1.1	1. Perfgraph (as measured) — annotated	1
1.2	2. Why ApplySpatialDerivative is the bottleneck	2
1.3	3. Why the kernel runs twice (the cheap $1.5\times$ before the hard work)	3
1.4	4. Optimization levers (prioritized)	3
1.5	5. Verification strategy (per the project constraints)	5
1.6	6. Sequencing & risk	6

1 Optimization Plan: ADER dynamic-rupture hot path (TPV31 spatial, $p \geq 2$ / ADER-O3)

Date: 2026-05-31 **Source perfgraph:** tpv31_p2_ader03_noflux_50m_flex.sbatch (TPV31_PERFGRAPH=1), matrix Riemann (BimaterialWaveOperator), ADER-O3, 50 m straight-tet mesh, flex queue. Caliper runtime-report(calc.inclusive=true). **Goal:** identify what makes the run slow and lay out a *numerics-preserving* optimization plan. **No code is changed by this document** — it is the plan; implementation + verification follow under the constraints in §6.

This is an *optimization* companion to caliper_perfgraph_plan.md (which was the plan that *added* the MFEM_PERF_SCOPE annotations that produced this graph). It does not revisit annotation placement.

1.1 1. Perfgraph (as measured) — annotated

Path	Min/rank	Max/rank	Avg/rank	Time %	notes
seas::spatial_dyn::step	1409.46	1409.84	1409.83	99.97	whole loop
seas::spatial_dyn::AdvanceADERWithSubStep	1408.60	1409.12	1408.99	99.91	
WaveOperator::ComputeADERSubStepStates	457.09	609.11	485.73	34.44	CK recursion #1
WaveOperator::ApplySpatialDerivative	449.07	600.84	476.67	33.80	<-- hot kernel
WaveOperator::AdvanceADER	790.75	825.82	807.25	57.24	
WaveOperator::ApplySpatialDerivative	448.96	600.46	476.58	33.80	<-- hot kernel (CK recursion #2)
WaveOperator::ComputeADERFaceFluxRHS	66.80	264.67	224.87	15.95	<-- 4x load imbalance
WaveOperator::ComputeADERVolumeUpdate	75.41	101.92	80.30	5.69	
WaveOperator::ComputeVolumeRHS	75.40	101.92	80.29	5.69	
WaveOperator::ComputeADERSharedFaceFluxRHS	9.44	25.80	18.30	1.30	
seas::spatial_dyn::friction_substep	0.03	0.40	0.16	0.01	negligible
BimaterialWaveOperator::ComputeMaxDt	(0.04 total)			0.00	negligible

Where the time actually goes (run totals)

Region	Avg s	% of run	Verdict
ApplySpatialDerivative (both subtrees: 476.67 + 476.58)	953.25	67.6 %	dominant — attack first
ComputeADERFaceFluxRHS	224.87	15.9 %	second; also $4\times$ load-imbalanced
ComputeVolumeRHS (inside ComputeADERVolumeUpdate)	80.29	5.7 %	same recompute pattern as the kernel
ComputeADERSharedFaceFluxRHS	18.30	1.3 %	minor; imbalanced

Region	Avg s	% of run	Verdict
friction_substep	0.16	0.01 %	irrelevant — do not optimize

Headline: $\sim \frac{2}{3}$ of wall time is one element-local kernel, `ApplySpatialDerivative`, run twice per step over the same input. The friction solve — the physically interesting part — is 0.01 %. This is a *bulk-wave propagation* cost problem, not a friction problem.

1.2 2. Why `ApplySpatialDerivative` is the bottleneck

`dynamic/wave_operator.inl:892`. Per call it computes, for every element and every state component, $dQ_c = M_e^{-1} K_d^e Q_c$ (element-local L2-projected $\partial_{\{x_d\}}$). The body (inner of the element loop):

```
const IntegrationRule &ir = IntRules.Get(fe->GetGeomType(), 2*order_); // per element, per call
Vector shape(ndof);          DenseMatrix dshape(ndof, 3);           // HEAP alloc per element, per call
for (int q = 0; q < nqp; q++) {
    Tr->SetIntPoint(&ip);
    real_t w = ip.weight * Tr->Weight(); // Jacobian determinant – geometry only
    fe->CalcShape(ip, shape);             // reference shape – geometry only
    fe->CalcPhysDShape(*Tr, dshape);      // physical  $\partial\varphi = \text{ref}\partial\varphi \cdot J^{-1}$  – geometry only, INVERTS J per QP
    ...  $\sum_j dshape(j, \text{dir}) Q_c(j)$  ... // the only Q-dependent work
}
const DenseMatrix &M_inv = elem_mass_inv_[e]; // already cached
... dQ_c = M_inv * KdQ_c ...
```

Root causes

1. **Everything geometric is recomputed every call.** `CalcShape`, `CalcPhysDShape` (which inverts the 3×3 Jacobian per quadrature point and multiplies by the reference gradient), and `Tr->Weight()` depend **only on the mesh**, not on Q or time. They are identical across all $6 \cdot (\text{order} - 1) = 12$ calls per macro-step (O3) and across every macro-step of the run. The mesh is a **straight (affine) tet** mesh (the `sbatch` header says so), so the Jacobian is even *constant within each element* — the per-QP `CalcPhysDShape` is doubly redundant.
2. **The operation is a fixed linear map.** $dQ_c = (M_e^{-1} K_d^e) Q_c$, and $D_d^e \equiv M_e^{-1} K_d^e$ is a constant $\text{ndof} \times \text{ndof}$ matrix per element per direction. The current code re-derives the action of K_d^e by quadrature on every call instead of applying a precomputed D_d^e .
3. **Per-element heap allocations** (`Vector shape`, `DenseMatrix dshape`) inside the `ne_` loop — `malloc/free` per element per call.
4. **It runs twice per step over identical inputs** (see §3): the CK Taylor recursion is executed independently by `ComputeADERSubStepStates` (`wave_operator.inl:1201`) and by `ComputeAD-ERTimeIntegrated` (called from `AdvanceADER`, `wave_operator.inl:5374`).

`ComputeVolumeRHS` (`wave_operator.inl:805`) and the PML loop in `AdvanceADER` (`wave_operator.inl:5395`) share root cause (1) — same per-QP `CalcShape/CalcPhysDShape/Weight` recompute, same per-element allocs.

Precedent for the fix already exists

`AssembleElementMassInverse()` (`wave_operator.inl:5552`) already precomputes a per-element dense operator (`elem_mass_inv_`, `std::vector<DenseMatrix>`, declared `wave_operator.hpp:748`)

once at setup using this exact element/QP loop. The derivative-operator cache mirrors it one-for-one.

1.3 3. Why the kernel runs twice (the cheap $1.5\times$ before the hard work)

Driver `AdvanceADERWithSubStep_Spatial` (`drivers/spatial_dyn_driver.cpp:375`), once per macro-step:

```

wave.ComputeADERSubStepStates(Q, dt_step, ader_order, tau_nodes, Q_per_node); // line 427
... per-substep fault QP evaluation + friction ...
wave.AdvanceADER(Q, dt_step, ader_order, Q_new); // line 473

```

Both calls take the **same Q , same dt_step , same $ader_order$** . Internally both build the identical Cauchy-Kovalevskaya sequence $D(0)=Q$, $D(k+1) = -\sum_d A_d \cdot \partial_{x_d} D(k)$:

- `ComputeADERSubStepStates` (`wave_operator.inl:1267–1306`) forms the nodal Taylor states $Q_per_node[o] = \sum_k (\tau_o^k/k!) D(k)$.
- `ComputeADERTimeIntegrated` (`wave_operator.inl:1146–1182`, via `AdvanceADER`) forms the time integral $I = \sum_k (dt^{k+1}/(k+1)!) D(k)$.

Both are linear combinations of the same $D(0..order-1)$ basis. The $D(k)$ recursion — i.e. *all* of `ApplySpatialDerivative` — is computed twice from identical inputs. Compute $D(k)$ once, form both outputs $\Rightarrow$ the second `ApplySpatialDerivative` subtree ($476.6\text{ s} \approx 33.8\%$) disappears. This is a pure restructuring with **byte-identical** numerics (same $D(k)$, same weights).

1.4 4. Optimization levers (prioritized)

Review guardrails (folded from `..._REVIEW.md`, R-001–R-004 — **MUST honor**). - **R-001 (CRITICAL, deadlock)**: `ComputeADERSubStepStates` / `Q_per_node` **MUST** stay **unconditional on every rank**. `Q_per_node` feeds a per-substep `ExchangeFaceNbrData` collective (`EvaluateBulkAtFaultQPsCanonical`, invariant R-1600) that every rank with face neighbours must enter — so the tempting “skip the substep recursion on fault-less ranks (most ranks at $np=800$)” is a **deadlock trap**. Lever 2 therefore **merges** the two recursions; it never skips recursion #1. - **R-002 (numerics)**: Lever 1 is **machine-eps**, **NOT bit-identical** end-to-end (the cached $M^{-1}K_d$ re-associates the round-off). Accept Lever 1 on **physical** metrics (rupture-front time, peak V , final slip within a stated tol), never bit-identity. Lever 2 is byte-identical **relative to a Lever-1-ON baseline** — its parity test must rebaseline after Lever 1. - **R-003 (numerics)**: the Lever-2 merge must keep **both** factorial-weight phases distinct (R-1503): $\tau^k/k!$ for the nodal states, $dt^{k+1}/(k+1)!$ for I . Do not unify the denominators. - **R-004 (ops)**: the Lever-1 cache must **fail loud or fall back** above a per-rank memory budget ($p4/dense$ partitions can OOM a node).

Lever 1 – Precompute per-element derivative operators $D_d^e = M_e^{-1} K_d^e$

Target: all `ApplySpatialDerivative` calls (67.6 %, both subtrees). **Idea:** at setup, alongside `elem_mass_inv_`, build `std::vector<std::array<DenseMatrix,3>>` `elem_deriv_op_` where `elem_deriv_op_[e][d]` is the $ndof \times ndof$ matrix s.t. $dQ_c = elem_deriv_op_[e][d] \cdot Q_c$. Construction

(in/after AssembleElementMassInverse, wave_operator.inl:5552):

```
K_d^e[i,j] =  $\sum_q w_q \cdot \varphi_i(x_q) \cdot \partial_d \varphi_j(x_q)$  // from CalcShape + CalcPhysDShape (setup only)
D_d^e      = elem_mass_inv_[e] · K_d^e // one DenseMatrix::Mult at setup
```

ApplySpatialDerivative hot body collapses to, per element per component: $dQ_c^e = D_d^e \cdot Q_c^e$ (one $\text{ndof} \times \text{ndof}$ mat-vec) — no CalcShape, no CalcPhysDShape, no Jacobian inversion, no quadrature loop, no per-call allocation. - **Batching (optional, extra win)**: gather the 9 components of an element into an $\text{ndof} \times \text{NUM_STATE}$ scratch and do one `DenseMatrix::Mult` ($D_d^e \cdot Q^e$) so the inner kernel is a BLAS gemm instead of 9 gemv. Requires a gather/scatter because the global layout is component-major ($Q[c \cdot \text{ndof_total} + \text{dof}]$); the gather cost is $\text{ndof} \cdot 9$ copies vs. the quadrature work it replaces — net win. Land the mat-vec form first, add batching only if reprofiling shows it matters. - **Orthogonal to the matrix/bimaterial path**. ApplyElementJacobian_ (the per-element A_d^e star-matrix apply, wave_operator.inl:1091) is a *separate* step after ApplySpatialDerivative and is untouched. D_d^e caches only the geometry/derivative part, which is identical for the scalar and bimaterial operators. So one change accelerates both WaveOperator and BimaterialWaveOperator (which inherits the base kernel). - **Memory cost**: $3 \cdot \text{ne_local} \cdot \text{ndof}^2 \cdot 8 \text{ B}$. p2 tet ($\text{ndof}=10$) $\approx 2.4 \text{ kB/elem}$; p3 tet ($\text{ndof}=20$) $\approx 9.6 \text{ kB/elem}$. At $\sim 7.5 \text{ k elem/rank}$ (6M tets / 800 ranks): $\sim 18 \text{ MB/rank}$ (p2), $\sim 72 \text{ MB/rank}$ (p3) $\rightarrow \leq \sim 3.6 \text{ GB/node}$ at 50 ranks/node on a 192 GB CLX node. Acceptable. (`elem_mass_inv_` is already stored, so the marginal memory is just the three K-derived operators.) **R-004 guard**: at cache-build, compute $3 \cdot \text{ne} \cdot \text{ndof}^2 \cdot 8 \text{ B}$ and **fail loud** (or fall back to the on-the-fly kernel) above a per-rank budget — p4 tet ($\text{ndof}=35$) $\approx 440 \text{ MB/rank}$ can OOM a node. - **Landing strategy (flagged in-place)**: a `DerivMode { OnTheFly, Cached }` toggle, **default OnTheFly** = today's quadrature kernel, so every byte-exact regression is unchanged until the flag is flipped; the cached path and cache are built only when `Cached` is selected. Both kernels stay reachable in one binary (required by the §5 equivalence test and the R-004 fallback). The new build logic lives in an isolated helper `dynamic/elem_derivative_cache.hpp`. - **Expected gain**: removes the per-QP `CalcPhysDShape/CalcShape/Weight` recompute — the bulk of the kernel — replacing it with a single cached mat-vec. Estimate **3–8× on the 67.6 %** (validate by reprofiling). - **Risk**: medium. `wave_operator.{hpp,inl}` are “extreme care” files (`miniapps/seas/CLAUDE.md`). The transform is exact for affine elements and algebraically exact in general (it caches the *same* $M^{-1}K_d$ the code computes today). Gate with the equivalence test in §5.

Lever 2 – Share the CK recursion between the two predictors

Target: the second ApplySpatialDerivative subtree (33.8 %). **Idea**: compute $D(0..order-1)$ once per macro-step and feed *both* the sub-step nodal states **and** the time-integrated predictor I . Concretely, either: - (a) extend `ComputeADERSubStepStates` to also accumulate I (treat the time-integral as one extra “node” with weights $dt^{k+1}/(k+1)!$), return it, and add an `AdvanceADER(Q, dt, order, Q_new, const Vector* I_precomputed)` overload that skips `ComputeADERTimeIntegrated` when I is supplied; or - (b) factor the shared recursion into a private `ComputeCKStates(Q, dt, order, ...)` that yields the $D(k)$ set (or directly both reductions), called once by the driver helper. - **Numerics (R-003)**: byte-identical — same $D(k)$, same Taylor/integral weights. The two factorial-phase comments (R-1503 at `wave_operator.inl:1283` and `:1161`) must be preserved: the recursion is shared, but the **two weight accumulations stay distinct** ($\tau^k/k!$ for nodes, $dt^{k+1}/(k+1)!$ for I , each with its own `fac/denom/k=0` term). Do NOT unify the denominators. - **R-001 (collective-safety)**: the merged routine runs **unconditionally on all ranks** — `Q_per_node` feeds the per-substep `ExchangeFaceNbrData` collective. Do NOT make it rank-conditional (fault-less skip $\Rightarrow$ deadlock).

Form both `Q_per_node` and `I` at the current `ComputeADERSubStepStates` call site, then pass `I` into the `AdvanceADER(..., I_precomputed)` overload; the collective ordering (substep exchange $\rightarrow$ corrector) is unchanged. - **Expected gain:** $\sim \frac{1}{2}$ of the *remaining* `ApplySpatialDerivative` cost after Lever 1. Compounds with Lever 1. - **Risk:** medium (touches the same extreme-care file + the driver). Smaller surface than Lever 1. Gate with the parity test in §5.

Levers 1 and 2 compound: 1 makes each kernel call cheap; 2 halves the number of calls. Together they target the full 67.6 %.

Lever 3 – ComputeVolumeRHS / PML loop: reuse cached geometry

Target: 5.7 % (`ComputeVolumeRHS`) + the PML loop in `AdvanceADER`. `ComputeVolumeRHS` (`wave_operator.inl:805`) has the identical `CalcShape/CalcPhysDShape/Weight` per-QP recompute and per-element `Vector/DenseMatrix` allocs. Cache the per-element/per-QP geometric factors (or, like Lever 1, precompute the volume operator) so the per-step cost drops to a few mat-vecs. Lower priority (smaller slice) but reuses Lever 1’s infrastructure. The PML loop (`:5395`) is inert for TPV31 (no PML) — skip unless a PML benchmark is profiled.

Lever 4 – ComputeADERFaceFluxRHS: imbalance + per-call overheads (15.9 %)

Target: the 224.9 s flux assembly with $4\times$ **load imbalance** (min 66.8 / max 264.7). Several distinct issues at `wave_operator.inl:3516`: 1. **Per-step MPI_Allreduce** (`:3539–3553`, the `has_bulk_bg_consensus` check, R-1505) runs on *every* `AdvanceADER`. It is a per-macro-step collective barrier that **absorbs upstream load imbalance** (ranks that finished the volume update early block here) — which is most of what the $4\times$ min/max spread is measuring. The check is a setup invariant: hoist it to run **once** (after `SetAbsorbingBackground`) and cache a verified flag. Removes a per-step barrier and de-noises the profile. 2. **std::getenv per call** (`:3597, :3625`) — `SEAS_TEST_EVAL_FACE_AVG_STAGE` and `SEAS_TEST_NONFAULT_BOTH_SYM` are read every step. Read once at setup, cache the parsed mode. Test-only hooks should not tax the production loop. 3. **GetFaceElementTransformations(f) + face geometry recomputed per call** over `mesh_.GetNumFaces()` (`:3637–3639`) — same “geometry is constant” observation as Lever 1, applied to faces. Caching face transforms/normals is a larger change; scope it only if (1)+(2) don’t bring the slice down. 4. **Genuine partition imbalance.** After (1) removes the synthetic barrier component, reprofile: if `ComputeADERFaceFluxRHS` is *still* $4\times$ imbalanced, the cause is uneven face/element distribution from the mesh partition. Investigate ParMETIS partition quality and whether fault-face ownership should weight the partition. This is a separate, larger work item — do not fold it into Levers 1–3.

Explicitly NOT worth optimizing

- `friction_substep` (0.01 %), `ComputeMaxDt` (~ 0 %), `ComputeADERSharedFaceFluxRHS` (1.3 %, and partly comm). Leave them.

1.5 5. Verification strategy (per the project constraints)

Constraint (memory feedback-no-local-mesh-runs): no full-mesh runs locally. All local verification = **compile + unit tests**; the end-to-end re-profile is a **Frontera** step. **Constraint (memory makefile-no-header-deps-stale-o):** `WaveOperator/BimaterialWaveOperator` are templates with bodies in `.inl`; the Makefile has no header deps $\rightarrow$ **make clean before building tests** (else `stale-.o`

ABI SIGABRT).

New unit tests (TDD – write the gate before the change)

1. **Lever 1 equivalence (R-002):** in `tests/unit/test_wave_operator_spatial_derivative.cpp`, with BOTH kernels reachable in one binary (DerivMode toggle), assert `CachedApplySpatialDerivative` matches `OnTheFly` to $\leq 1e-12$ **relative** on a small multi-element tet mesh, for each `dir` $\in \{0,1,2\}$ and a random `Q`. This is the primary correctness gate for Lever 1.
 - **NOT bit-identical end-to-end (R-002):** the cached $M^{-1}K_d$ re-associates round-off, so a full run's traces will differ at more than bit level. End-to-end acceptance is therefore on **physical** metrics (rupture-front arrival, peak v , final slip within a stated %), NOT bit-identity — do not assert bit-equality for any `Cached` build.
2. **Lever 2 parity (byte-identical, R-002 baseline):** assert the shared-recursion path produces `Q_new` and `Q_per_node` **bit-for-bit** identical to the separate `ComputeADERSubStepStates + AdvanceADER`, on a small mesh at orders $\{2,3,4\}$. **Baseline against a Lever-1-ON build** (same `DerivMode`) — comparing against a pre-Lever-1 reference would falsely fail on Lever-1 round-off. Lever 2 changes only *which object* computes $D(k)$, so at a fixed `DerivMode` it must not change a single bit.
3. **Bimaterial path:** run the equivalence/parity at least once through `BimaterialWaveOperator` so the matrix `ApplyElementJacobian_` interaction is covered (it should be untouched, but prove it).

Existing regression set (must stay green; same as the annotation plan)

`seas_test_wave_operator`, `seas_test_wave_operator_spatial_derivative`, `seas_test_bimaterial_wave_operator_parity`, `seas_test_ader_ck_predictor`, `seas_test_ader_linear_wave_equivalence`, `seas_test_ader_tpv102_smoke`. (See `caliper_perfgraph_plan.md` “Local Unit-Test Verification” for the exact worktree build invocation: `MFEM_DIR=../../ MFEM_BUILD_DIR=$MAIN ...`)

Frontera re-profile (acceptance)

Rebuild with Caliper (make `clean` first — empty-perfgraph hazard, see `README_caliper_build_and_run.md`) and re-run the `tpv31_p2_ader03` perfgraph. **Accept** when: (a) `ApplySpatialDerivative` total drops to a small fraction of `AdvanceADER`; (b) `ApplySpatialDerivative` appears under only *one* subtree (Lever 2); (c) total step time falls materially; (d) on-fault station traces (`tpv31_*.dat`) still match `tpv31/benchmark_data/sccec_{eqdyna,seisol}/` via `tpv31/visualize_results.py` (numerics unchanged). Pull artifacts with `jobs/tpv31_spatial/sync_results.sh`.

1.6 6. Sequencing & risk

Step	Lever	Expected	Risk	Numerics
1	Lever 1 (precompute D_d^e)	3–8× on 67.6 %	med	exact / machine-eps
2	Lever 2 (share CK recursion)	halve remaining kernel	med	byte-identical

Step	Lever	Expected	Risk	Numerics
3	Lever 4 (1)+(2) (hoist Allreduce + getenv)	de-noise 15.9 %, drop a barrier	low	identical
4	Lever 3 (volume geometry reuse)	~5.7 %	low-med	exact
5	Lever 4 (4) (partition balance)	recover the $4\times$ tail	high (separate effort)	identical

Do them in order and **reprofile between steps** — the relative shares shift as each lands, and Lever 1 may change which region is next-dominant. Land Lever 1 alone first (largest, self-contained) and validate end-to-end before stacking 2.

Cross-cutting risks

- **Extreme-care files** (`wave_operator.{hpp,inl}`, `drivers/spatial_dyn_driver.cpp`): every change needs the full unit-test gate above + `git diff` review. No “while I’m here” edits.
- **Stale-`.o` SIGABRT** (template + no header deps): mandatory `make clean` before each verification build.
- **Do not optimize away a correctness guard.** The R-1505 MPI_Allreduce (Lever 4.1) is a real consensus check — *move it to setup*, do not delete it; it must still fail loud if a rank skipped `SetAbsorbingBackground`.
- **Affine-only assumption** in Lever 1’s “constant within element” remark applies to the straight-tet TPV31 mesh; the $D_d^e = M^{-1}K_d$ caching itself is exact for curved elements too (it caches the same integrated operator), so no curved-mesh regression — but keep the homogeneous-order `ndof == ndof_per_el_guard` (`wave_operator.inl:933`).
- **Memory** (Lever 1): note the per-rank $3 \cdot n_e \cdot ndof^2$ budget in the build log; fail loud / fall back if a future very-high-order or very-dense partition blows the node memory.

Non-goals

Friction-solver tuning (0.01 %), I/O, the RK path (not used by TPV31), and algorithm changes to ADER order/CFL. This plan is strictly about removing redundant *bulk-wave* compute while holding numerics fixed.